Лабораторная работа № 12

Тема: Управление программным обеспечением в Linux: работа с пакетным менеджером APT, утилитой dpkg, репозиториями и аудит обновлений.

Цель работы: Закрепить практические навыки управления программным обеспечением в Debian-подобных системах. Научиться безопасно обновлять индексы репозиториев и пакетную базу (apt), анализировать метаданные и зависимости перед установкой (apt show), устанавливать и удалять ПО, работать с локальными пакетами через dpkg, решать проблемы нарушенных зависимостей (apt -f install), а также проводить пост-проверку результатов установки.

Необходимое ПО

- Эмулятор терминала для подключения по SSH.
- Развернутый Linux-сервер (Ubuntu/Debian) с доступом в Интернет.

Ситуация

Вы работаете системным администратором. Руководство поставило задачу развернуть на сервере дополнительное прикладное ПО и провести плановый аудит безопасности системы:

1. Синхронизировать локальный индекс пакетов с официальными зеркалами и безопасно обновить критические компоненты ОС.
2. Провести предварительный аудит утилиты сетевой диагностики (например, nmap или tmux), изучить её зависимости и только после этого установить в систему.
3. Установить специфический внутренний инструмент компании, поставляемый в виде изолированного локального файла .deb, столкнуться с проблемой отсутствующих в системе библиотек и штатно восстановить работоспособность пакетного менеджера.

Требования к выполнению работы

1. **Разделение операций:** Категорически запрещено путать команды apt update и apt upgrade. Они должны выполняться последовательно и осознанно.
2. **Принцип контроля зависимостей:** Перед установкой любого пакета из репозитория администратор обязан изучить его описание и список связанных библиотек.

3. **Безопасность автоматического удаления:** При вызове команды autoremove студент обязан внимательно прочитать список удаляемых пакетов, чтобы случайно не стереть нужные системные зависимости.

Порядок выполнения работы

Этап 1. Актуализация индексов и аудит пакетов перед установкой

1. Подключитесь к серверу по SSH под своей учетной записью с правами sudo.
2. Обновите локальный индекс пакетов (список доступного в репозиториях ПО), не изменяя сами программы на диске:

```
sudo apt update
```

Обратите внимание на строки вывода: зафиксируйте, с каких официальных репозиториев (зеркал) сервер скачивает индексы.

3. Проверьте информацию о пакете **nmap** (или tmux) до его установки с помощью команды:

```
apt show nmap
```

Найдите в выводе и зафиксируйте в отчете: версию пакета, его размер и строку Depends (зависимости пакета).

Этап 2. Установка, проверка версии и удаление пакета из репозитория

1. Установите пакет, прошедший аудит на предыдущем шаге:

```
sudo apt install nmap -y
```

2. Убедитесь, что программа установлена корректно и пакетный менеджер зафиксировал её в системе. Проверьте статус пакета через низкоуровневую утилиту dpkg:

```
dpkg -l | grep nmap
```

(В начале строки должен быть флаг ii, означающий «успешно установлен»).

3. Проверьте физическую версию установленной бинарной программы напрямую: nmap --version.
4. Симулируйте удаление программы, но с сохранением её конфигурационных файлов в системе (мягкое удаление):

```
sudo apt remove nmap -y
```

5. Проверьте статус пакета через dpkg -l | grep nmap. Флаги должны смениться на rc (пакет удален, но конфигурация осталась).
6. Полностью очистите систему от остатков пакета и его конфигураций: sudo apt purge nmap -y.

Этап 3. Работа с локальными пакетами (dpkg) и симуляция поломки зависимостей

1. Скачайте в домашний каталог любой простой .deb пакет из сети, у которого заведомо есть внешние зависимости (например, утилиту gdebi-core или conky для вашей версии ОС, либо используйте пакет, подготовленный преподавателем), используя команду wget.
2. Попробуйте установить этот локальный пакет напрямую через утилиту dpkg, которая, согласно лекции, не умеет самостоятельно скачивать зависимости из Интернета:

sudo dpkg -i имя_пакета.deb

3. Зафиксируйте падение команды с ошибкой зависимостей (Dependency problems prevent configuration of...).
4. Попробуйте запустить любую другую команду пакетного менеджера, например sudo apt update. Убедитесь, что база данных пакетов заблокирована или выдает предупреждение о «сломанных» пакетах в системе.

Этап 4. Исправление нарушенных зависимостей

1. Используя специальный флаг утилиты apt из лекции, заставьте систему автоматически проанализировать поломку, скачать недостающие библиотеки из официальных репозиториев и донастроить наш локальный пакет:

sudo apt -f install -y

2. Убедитесь, что команда завершилась успешно. Повторно проверьте статус пакета через dpkg -l, чтобы подтвердить, что статус сменился на ii (установлен).

Этап 5. Управление изолированными пакетами Snap (Новое задание)

1. Выполните поиск доступных версий утилиты (например, популярного диспетчера процессов htop) в магазине изолированных пакетов Snap:

```
snap find htop
```

2. Установите программу htop (или текстовый редактор) через подсистему Snap:

```
sudo snap install htop
```

3. Выведите список всех установленных в системе snap-пакетов, чтобы убедиться, что программа зафиксирована в изолированной среде:

```
snap list
```

4. Обратите внимание на вывод команды. Сравните его с выводом dpkg -l. В отчете ответьте на вопрос: видны ли программы из snap list внутри стандартной базы данных dpkg?

5. Удалите установленный snap-пакет:

```
sudo snap remove htop
```

Этап 6. Плановое обновление системы (apt upgrade)

1. Запустите процесс безопасного обновления уже установленных в системе пакетов до их актуальных версий:

```
sudo apt upgrade -y
```

2. Внимательно изучите финальный вывод команды. Зафиксируйте, потребовалось ли системе обновление ядра или уведомление о необходимости перезагрузки сервера (наличие файла /var/run/reboot-required).

В присутствовать:

1. **Скриншот №1 (Этап 1):** Экран вывода команды apt show nmap, где четко видны строки описания пакета, его версия и список зависимостей.

2. **Скриншот №2 (Этап 2):** Вывод команды dpkg -l | grep nmap сразу после выполнения apt remove, подтверждающий переход пакета в статус сохранения конфигурации rc.

3. **Скриншот №3 (Этап 3):** Момент падения утилиты dpkg -i при попытке установить локальный пакет, с отображением текста об ошибке неудовлетворенных зависимостей.
4. **Скриншот №4 (Этап 4):** Успешное выполнение команды sudo apt -f install, демонстрирующее, что менеджер пакетов скачал недостающие файлы и исправил конфигурацию системы.
5. **Скриншот №5 (Этап 5):** Финальные строки вывода команды sudo apt upgrade, показывающие общую статистику обновленных пакетов в операционной системе.

Контрольные вопросы:

1. Что представляет собой пакет в ОС Linux (на примере формата .deb)? Какие три основные категории данных (компонента) он содержит внутри себя?
2. Что такое репозиторий программного обеспечения? Какую роль выполняет локальный «индекс пакетов» на сервере, и почему команда apt update не обновляет приложения на диске?
3. В чем заключается ключевое практическое различие между командами apt update и apt upgrade? К каким последствиям может привести выполнение второй команды без первой?
4. Почему низкоуровневая утилита dpkg не способна самостоятельно разрешить зависимости при установке локального файла .deb? Какую роль в этот момент берет на себя высокоуровневый менеджер apt?
5. Объяснить разницу между командами apt remove и apt purge. В каком файле или каталоге системы можно отследить, остались ли конфигурационные файлы программы после её удаления?
6. Как работает команда sudo apt -f install? Какую системную неисправность («поломку») пакетного менеджера она призвана устранять?
7. Что такое пакеты формата snap (или flatpak), и в чем заключается их принципиальное архитектурное отличие от классических .deb пакетов с точки зрения изоляции и управления зависимостями?
8. Какие основные риски безопасности и стабильности несет в себе подключение сторонних (PPA или неофициальных) репозиториев

на корпоративных серверах приложений? Какие правила проверки доверия к источнику должен соблюдать сисадмин?